Ministerul Educaţiei, Culturii şi Cercetării al Republicii Moldova
Universitatea Tehnică a Moldovei
Departamentul Ingineria Software și Automatică

RAPORT

Lucrare de laborator Nr. 1-2
Disciplina: Sisteme de Operare (Operating Systems)

A efectuat: Kushnirenko Ecaterina st.gr. , FAF-243

A verificat : prof.univ.
Dumitrascu Marius

Chișinău 2026

INTRODUCTION

Objective:

The objective of this laboratory work is to observe the operating system in its role of resource manager, first from the outside on a real Linux system and then from the inside on xv6, a small teaching Unix. The work covers the four core resources an OS manages (files, processes, memory and devices) and the boundary between user programs and the kernel, which is crossed through system calls.

Task:

Laboratory work 1 (Chapter 2, Linux): explore the file tree and permissions, list, start and stop processes and inspect them through /proc, examine RAM and swap usage, and look at block devices and mounted file systems, recording observations for each of the four resources.

Laboratory work 2 (Chapter 3, xv6): build xv6 from source and boot it under QEMU, use its built-in programs, read the source to find where system calls are declared and implemented, and write a first user program, `sleep`, that calls into the kernel.

Both labs were carried out on a personal laptop running Arch Linux (kernel 7.2.6-arch2-1, x86_64, 31 GiB of RAM, 1 TB NVMe SSD) instead of the provided virtual machine, so the results describe a real desktop system with a graphical session and everyday applications running.

1 THEORETICAL CONSIDERATIONS

An operating system is the layer of software that stands between application programs and the hardware. It turns a raw disk into a tree of named files with owners and permissions, turns programs into isolated processes that it schedules on the CPUs, divides physical memory between those processes and backs it with swap, and hides every disk, terminal and peripheral behind a uniform, file-like interface. In UNIX-like systems this last idea is summarised as “everything is a file”.

The kernel runs in a privileged CPU mode with full access to hardware and memory, while user programs run in an unprivileged mode inside their own address spaces. A program that needs a resource (reading a file, creating a process, allocating memory) must ask the kernel through a system call such as `open`, `read`, `write`, `fork` or `exit`. Linux exposes a large amount of kernel state through pseudo-file systems such as /proc and /sys, which lets the manager be observed with ordinary tools. xv6 is a re-implementation of Unix Version 6 in about 9,000 lines of C for RISC-V; because it is so small, both sides of the system call boundary (the program in user/ and the kernel code in kernel/) can be read in full [2], [3].

2 THE OS FROM THE OUTSIDE: LINUX AS A RESOURCE MANAGER

This chapter corresponds to Laboratory work 1 [1]. The four resources managed by the OS were observed live in the terminal, and the answers to the questions of each part are given after the output.

2.1 Files and directories (file management)

A working folder was created, the machine was identified, and the root of the file tree was listed. Then a file was created, copied, renamed and removed, and its permissions were changed.

```
[nnorian@nnorian ~]$ mkdir -p ~/os-lab1 && cd ~/os-lab1
[nnorian@nnorian os-lab1]$ whoami
nnorian
[nnorian@nnorian os-lab1]$ uname -a
Linux nnorian 7.2.6-arch2-1 #1 SMP PREEMPT_DYNAMIC Mon, 14 Sep 2026 22:41:30 +0000
x86_64 GNU/Linux
[nnorian@nnorian os-lab1]$ uptime
 15:50:33 up 1 day, 22:52,  1 user,  load average: 0.33, 0.60, 0.60
[nnorian@nnorian os-lab1]$ ls -la /
lrwxrwxrwx   1 root root     7 Oct 12  2025 bin -> usr/bin
drwxr-xr-x   4 root root  4096 Jan  1  1970 boot
drwxr-xr-x  21 root root  4340 Sep 26 16:58 dev
drwxr-xr-x  99 root root  4096 Sep 28 15:40 etc
drwxr-xr-x   3 root root  4096 Oct 30  2025 home
lrwxrwxrwx   1 root root     7 Oct 12  2025 lib -> usr/lib
dr-xr-xr-x 488 root root     0 Sep 26 16:58 proc
drwx------   8 root root  4096 Apr 28 08:50 root
dr-xr-xr-x  13 root root     0 Sep 26 16:58 sys
drwxrwxrwt  25 root root   560 Sep 28 15:47 tmp
drwxr-xr-x   9 root root  4096 Sep 23 20:16 usr
drwxr-xr-x  14 root root  4096 Sep 25 22:31 var
   ... (output shortened)
```

Figure 2.1 System identification and the root of the file tree

```
[nnorian@nnorian os-lab1]$ mkdir demo && cd demo
[nnorian@nnorian demo]$ echo "hello operating systems" > note.txt
[nnorian@nnorian demo]$ cp note.txt copy.txt
[nnorian@nnorian demo]$ mv copy.txt renamed.txt
[nnorian@nnorian demo]$ ls -l
-rw-r--r-- 1 nnorian nnorian 24 Sep 28 15:52 note.txt
-rw-r--r-- 1 nnorian nnorian 24 Sep 28 15:52 renamed.txt
[nnorian@nnorian demo]$ rm renamed.txt
[nnorian@nnorian demo]$ chmod 600 note.txt
[nnorian@nnorian demo]$ ls -l note.txt
-rw------- 1 nnorian nnorian 24 Sep 28 15:52 note.txt
[nnorian@nnorian demo]$ chmod 644 note.txt
```

Figure 2.2 Creating, copying, renaming, removing a file and changing its permissions

Observations. The files created in the lab are owned by the user `nnorian` and belong to the group `nnorian`. After `chmod 600`, the ten characters `-rw-------` describe note.txt as follows: the first character `-` means a regular file; `rw-` means the owner may read and write it; the two remaining `---` triplets mean that the group and all other users have no access at all.

Several directories directly under / were identified:

- `/etc` holds system-wide configuration files;
- `/dev` holds device files, through which hardware is accessed;
- `/proc` is a virtual file system that shows live kernel and process information (its size is 0);
- `/home` holds the users' personal folders (documents, repositories, photos);
- `/boot` holds the kernel image and bootloader files needed to start the laptop.

It is also visible that on Arch Linux `/bin`, `/sbin`, `/lib` and `/lib64` are only symbolic links into `/usr`, and that `/root` is readable only by the root user (`drwx------`).

2.2 Processes (process management)

The processes managed by the OS were listed and counted, a background process was started, found and terminated, and the kernel's view of a process was read from /proc.

```
[nnorian@nnorian ~]$ ps aux | head -4
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0  20752 14944 ?        Ss   Sep26   0:04
/usr/lib/systemd/systemd --switched-root ...
root           2  0.0  0.0      0     0 ?        S    Sep26   0:00 [kthreadd]
root           3  0.0  0.0      0     0 ?        S    Sep26   0:00
[pool_workqueue_release]
[nnorian@nnorian ~]$ ps aux | wc -l
483
[nnorian@nnorian ~]$ top
top - 15:55:59 up 1 day, 22:57,  1 user,  load average: 0.34, 0.50, 0.57
Tasks: 489 total, 1 running, 487 sleep, 0 d-sleep, 0 stopped, 1 zombie
%Cpu(s):  0.6 us,  0.4 sy,  0.0 ni, 98.5 id,  0.0 wa,  0.3 hi,  0.1 si,  0.0 st
MiB Mem :  31195.2 total,  11800.5 free,  12800.8 used,   7726.5 buff/cache
MiB Swap:   4096.0 total,   4096.0 free,      0.0 used.  18394.4 avail Mem
    PID USER      PR  NI    VIRT    RES    SHR S  %CPU  %MEM     TIME+ COMMAND
  75684 nnorian   20   0 1181508 197468 155432 S   3.6   0.6   0:09.55 kgx
   2875 nnorian   20   0 4116744 288032 143772 S   2.7   0.9  42:30.90 gnome-shell
  64328 nnorian   20   0 2146996 933080 302464 S   0.7   2.9   3:16.74 Telegram
   5669 root      20   0 2684032 163928  58264 S   0.3   0.5   2:57.13 dockerd
   7128 nnorian   20   0   53.1g 613412 381492 S   0.3   1.9  20:27.92 chrome
      1 root      20   0   20752  14944  10484 S   0.0   0.0   0:04.69 systemd
   ... (output shortened)
```

Figure 2.3 Listing and counting the running processes

```
[nnorian@nnorian ~]$ sleep 300 &
[1] 78010
[nnorian@nnorian ~]$ jobs
[1]+  Running                 sleep 300 &
[nnorian@nnorian ~]$ ps -ef | grep sleep
nnorian    78010   77839  0 15:56 pts/2    00:00:00 sleep 300
nnorian    78146   77839  0 15:56 pts/2    00:00:00 grep --color=auto sleep
[nnorian@nnorian ~]$ kill 78010
[nnorian@nnorian ~]$ sleep 300 &
[2] 78209
[1]   Terminated              sleep 300
[nnorian@nnorian ~]$ PID=$!
[nnorian@nnorian ~]$ ls /proc/$PID/
arch_status  cgroup  cmdline  comm  cwd  environ  exe  fd  io  limits  maps  mem
mounts  net  ns  oom_score  root  sched  smaps  stack  stat  statm  status  task ...
[nnorian@nnorian ~]$ cat /proc/$PID/status | head -20
Name:   sleep
State:  S (sleeping)
Tgid:   78209
Pid:    78209
PPid:   77839
Uid:    1000    1000    1000    1000
Gid:    1000    1000    1000    1000
Groups: 150 960 991 998 1000
VmPeak:     6016 kB
VmSize:     6016 kB
   ... (output shortened)
[nnorian@nnorian ~]$ kill $PID
```

Figure 2.4 Starting, finding and stopping a background process and reading /proc/<pid>/status

Observations. Process number 1 is `systemd` (`/usr/lib/systemd/systemd`). It is the init system: the first user-space process started by the kernel, which then starts and supervises all other services, and which becomes the parent of orphaned processes.

About 482 processes were running (`ps aux | wc -l` returned 483, minus one line for the header). The number is high because this is a personal laptop with a desktop session and applications such as Chrome, Telegram, Docker and Obsidian open, plus many kernel threads (the names in square brackets). `top` reported 489 tasks because it was sampled a moment later; processes appear and disappear constantly.

For the sleeping process, the State line in /proc/<pid>/status reads `S (sleeping)`: the process is blocked waiting for its timer and does not use the CPU. The `PPid` field (77839) is the PID of the shell that started it, showing the parent–child relation between processes.

Interesting output. `top` showed one zombie process: a process that has already exited but whose parent has not yet collected its exit status with `wait()`. The shell also printed `[1] Terminated sleep 300` only at the next prompt, after the `kill`, because this is when it collected the status of its finished child.

2.3 Memory (memory management)

```
[nnorian@nnorian ~]$ free -h
               total        used        free      shared  buff/cache   available
Mem:            30Gi        12Gi        11Gi       623Mi       7.6Gi        17Gi
Swap:          4.0Gi          0B       4.0Gi
[nnorian@nnorian ~]$ cat /proc/meminfo | head -6
MemTotal:       31943852 kB
MemFree:        11966108 kB
MemAvailable:   18724236 kB
Buffers:          397424 kB
Cached:          7236192 kB
SwapCached:            0 kB
[nnorian@nnorian ~]$ sleep 300 & PID=$!
[1] 78734
[nnorian@nnorian ~]$ grep VmRSS /proc/$PID/status
VmRSS:      4252 kB
[nnorian@nnorian ~]$ kill $PID
```

Figure 2.5 RAM, swap and the resident memory of a single process

Observations. The laptop has 30 GiB of RAM in total (MemTotal = 31,943,852 kB). About 11 GiB is completely free, but 17 GiB is available, because the 7.6 GiB used as buffer and page cache can be reclaimed by the kernel whenever a process needs it.

Swap is space to which the kernel moves memory pages that are not currently in use when RAM runs low, so that active processes can keep running. The system has 4 GiB of swap, none of it used. Interestingly, `lsblk` shows that this swap is `zram0`: a compressed block device kept in RAM itself rather than a partition on the SSD. Pages are compressed instead of being written to disk, which is much faster.

A bare `sleep` process has a resident set (VmRSS) of 4,252 kB (about 4 MB), while its virtual size (VmSize) is 6,016 kB. This was surprising for a program that does nothing, but most of this memory is the shared C library and dynamic loader mapped into the process; those pages are shared with every other process that uses them, so the real extra cost of one more `sleep` is much smaller.

2.4 Devices and storage (I/O management)

```
[nnorian@nnorian ~]$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/nvme0n1p2  937G  452G  438G  51% /
devtmpfs         16G     0   16G   0% /dev
tmpfs            16G   47M   16G   1% /dev/shm
tmpfs           6.1G  2.0M  6.1G   1% /run
tmpfs            16G  2.9M   16G   1% /tmp
/dev/nvme0n1p1 1022M   66M  957M   7% /boot
tmpfs           3.1G  241M  2.9G   8% /run/user/1000
   ... (output shortened)
[nnorian@nnorian ~]$ lsblk
NAME        MAJ:MIN RM   SIZE RO TYPE MOUNTPOINTS
zram0       253:0    0     4G  0 disk [SWAP]
nvme0n1     259:0    0 953.9G  0 disk
├─nvme0n1p1 259:1    0     1G  0 part /boot
└─nvme0n1p2 259:2    0 952.9G  0 part /
[nnorian@nnorian ~]$ du -sh ~/os-lab1
12K     /home/nnorian/os-lab1
[nnorian@nnorian ~]$ ls -l /dev | head -5
crw-r--r--   1 root    root       10,   235 Sep 26 16:58 autofs
drwxr-xr-x   2 root    root             120 Sep 26 16:58 block
crw-rw----   1 root    disk       10,   234 Sep 26 16:58 btrfs-control
crw-------   1 root    root        5,     1 Sep 26 16:58 console
[nnorian@nnorian ~]$ mount | head -3
/dev/nvme0n1p2 on / type ext4 (rw,relatime)
devtmpfs on /dev type devtmpfs
(rw,nosuid,size=15896188k,nr_inodes=3974047,mode=755,...)
tmpfs on /dev/shm type tmpfs (rw,nosuid,nodev,inode64,huge=advise,usrquota)
```

Figure 2.6 Mounted file systems, block devices and device files

Observations. The root file system / is mounted from `/dev/nvme0n1p2`, formatted as ext4. The name reads as: `nvme` – an NVMe (Non-Volatile Memory Express) SSD, `0` – the first NVMe controller, `n1` – namespace 1, `p2` – partition 2. The first partition, `nvme0n1p1` (1 GiB), is mounted on /boot.

One entry from /dev: `/dev/nvme0n1` stands for the whole physical NVMe SSD of the laptop (953.9 GiB). Another, `/dev/console`, is a character device (the leading `c`) that stands for the system console.

“Everything is a file” means that disks, partitions, terminals, the console and even live kernel and process data all appear as paths in one tree, which can be listed with `ls` and read with `cat` like ordinary files, even though not all of them contain human-readable text.

3 THE OS FROM THE INSIDE: BUILDING xv6

This chapter corresponds to Laboratory work 2 [1]. xv6 [2] was built from source, booted under QEMU, used from its own shell, read at source level and extended with a new user program.

3.1 Building and booting xv6

The xv6-riscv repository [3] was placed in the course folder `os-course/xv6-riscv`. Its top level contains `kernel/` (the operating system), `user/` (the programs that run on it), `mkfs/` (the tool that builds the file system image), the `Makefile` and the `README` (Figures 3.1 and 3.2).

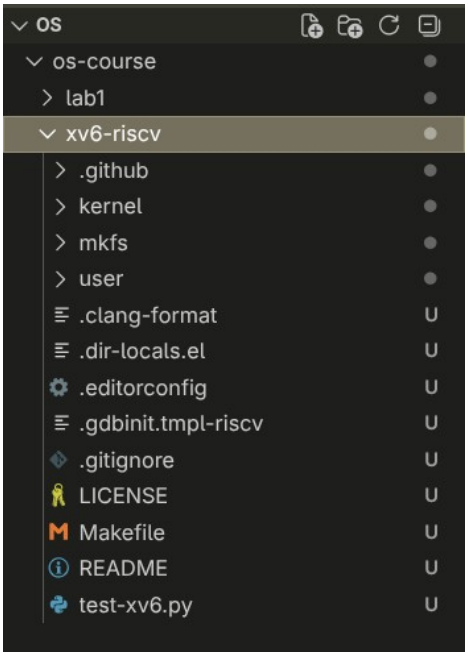


Figure 3.1 Structure of the xv6-riscv source tree in the editor

```
[nnorian@nnorian xv6-riscv]$ ls
kernel  LICENSE  Makefile  mkfs  README  test-xv6.py  user
[nnorian@nnorian xv6-riscv]$
```

Figure 3.2 Top-level contents of the xv6-riscv folder

The kernel and user programs were compiled with the `riscv64-elf-gcc` cross-compiler and started with `make qemu` (Figure 3.3). The two messages about `bc` at the beginning are harmless: one of the checks in the Makefile calls the `bc` calculator, which is not installed on the laptop, so that check fails silently while the compilation itself continues normally.

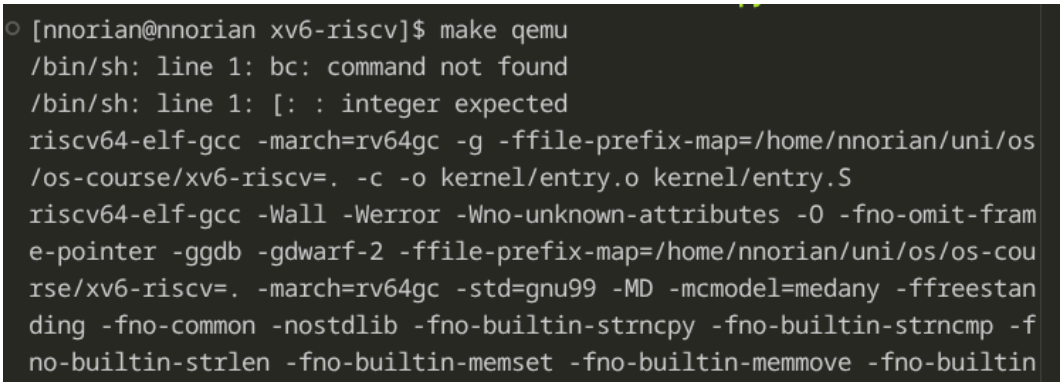


Figure 3.3 Compiling xv6 with the RISC-V cross-compiler (make qemu)

QEMU emulates a RISC-V `virt` machine with 128 MB of RAM and three harts (hardware threads, `-smp 3`). The kernel booted, the two secondary harts reported starting, `init` started the shell, and the familiar `$` prompt appeared. `ls` shows each file with its type (1 = directory, 2 = regular file, 3 = device), its inode number and its size in bytes (Figure 3.4).

```
xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2441
cat            2 3 38128
echo           2 4 36912
forktest       2 5 18264
grep           2 6 41688
init           2 7 37296
kill           2 8 36848
ln             2 9 36632
ls             2 10 40256
mkdir          2 11 36896
rm             2 12 36888
sh             2 13 60840
stressfs       2 14 37768
usertests      2 15 209576
grind          2 16 53384
wc             2 17 39160
zombie         2 18 36160
logstress      2 19 38920
forphan        2 20 37680
dorphan        2 21 37112
sync           2 22 36280
console        3 23 0
$ cat README
xv6 is a re-implementation of Dennis Ritchie's and Ken Thompson's Unix
Version 6 (v6).  xv6 loosely follows the structure and style of v6,
but is implemented for a modern RISC-V multiprocessor using ANSI C.
   ... (output shortened)
```

Figure 3.4 Booting xv6 and listing its file system

3.2 Using xv6 as a Unix

The built-in programs `echo`, `grep` and `wc` were run, including a pipe between two programs (Figure 3.5). `ls | grep c` kept only the entries whose line contains the letter c, and `wc README` reported 48 lines, 336 words and 2,441 bytes, which matches the size of README shown by `ls`. QEMU was then closed with the Ctrl-A, X sequence (Figure 3.6).

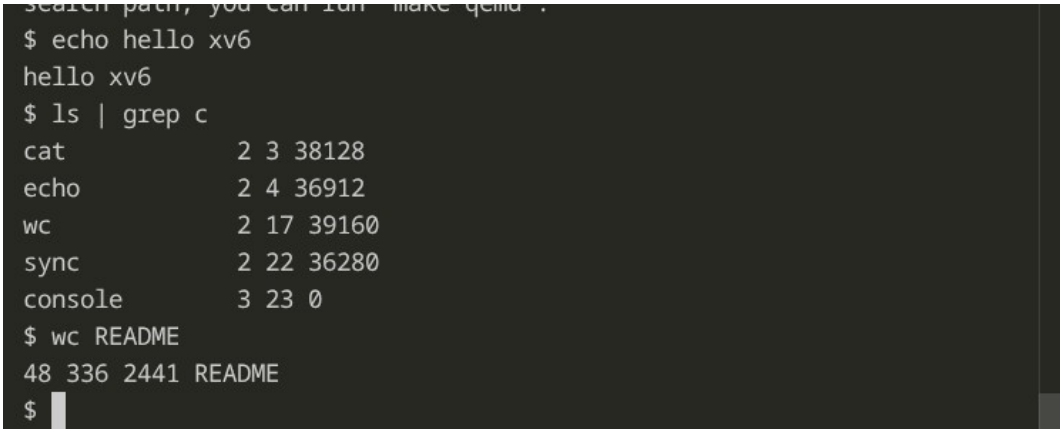


Figure 3.5 Running echo, a pipe (ls | grep c) and wc inside xv6



Figure 3.6 Leaving the emulator with Ctrl-A, X

Observations. (1) Three programs shipped with xv6 are `cat`, `grep` and `wc` (others include `ls`, `echo`, `mkdir`, `rm`, `kill` and `sh`). Even the terminal is present as a file: `console`, of type 3 (device).

(2) For the pipe `ls | grep c` to work, the OS must provide **processes** – the shell uses `fork()` and `exec()` to run `ls` and `grep` at the same time as two separate processes – and **inter-process communication** – the `pipe()` system call creates a kernel buffer with a write end and a read end, and file descriptors (rearranged with `close()` and `dup()`) connect the standard output of the first process to the standard input of the second.

(3) The xv6 shell looks and behaves like the Linux shell from Chapter 2 (the same commands, pipes and redirection), but it is far more minimal: there is no history, tab completion, job control, environment variables or scripting, and `ls` shows only type, inode and size instead of permissions and owners.

3.3 Reading the source

The whole OS is split into two folders: `kernel/`, which is the operating system itself (scheduler, memory, file system, system-call handlers) and runs privileged, and `user/`, which holds the ordinary programs that run on top of it. Two user programs were read first. `user/ls.c` (Figures 3.7 and 3.8) opens the path with `open()` and inspects it with `fstat()`; `user/cat.c` (Figures 3.8 and 3.9) is a loop that calls `read()` into a 512-byte buffer and `write()` to file descriptor 1 until `read()` returns 0.

```
[nnorian@nnorian xv6-riscv]$ sed -n "1,40p" user/ls.c
#include "kernel/types.h"
#include "kernel/stat.h"
#include "user/user.h"
#include "kernel/fs.h"
#include "kernel/fcntl.h"

char *
fmtname(char *path)
{
  static char buf[DIRSIZ + 1];
  char *p;

  // Find first character after last slash.
  for (p = path + strlen(path); p >= path && *p != '/'; p--)
    ;
  p++;

  // Return blank-padded name.
  if (strlen(p) >= DIRSIZ)
    return p;
  memmove(buf, p, strlen(p));
  memset(buf + strlen(p), ' ', DIRSIZ - strlen(p));
  buf[sizeof(buf) - 1] = '\0';
  return buf;
}

void
ls(char *path)
{
  char buf[512], *p;
  int fd;
  struct dirent de;
  struct stat st;
```

Figure 3.7 Beginning of user/ls.c

```
[nnorian@nnorian xv6-riscv]$ sed -n "1,40p" user/ls.c
  struct dirent de;
  struct stat st;

  if ((fd = open(path, O_RDONLY)) < 0) {
    fprintf(2, "ls: cannot open %s\n", path);
    return;
  }

  if (fstat(fd, &st) < 0) {
[nnorian@nnorian xv6-riscv]$
[nnorian@nnorian xv6-riscv]$ sed -n "1,30p" user/cat.c
#include "kernel/types.h"
#include "kernel/fcntl.h"
#include "user/user.h"

char buf[512];

void
cat(int fd)
{
  int n;

  while ((n = read(fd, buf, sizeof(buf))) > 0) {
    if (write(1, buf, n) != n) {
      fprintf(2, "cat: write error\n");
      exit(1);
    }
  }
  if (n < 0) {
    fprintf(2, "cat: read error\n");
    exit(1);
  }
}
```

Figure 3.8 End of the ls.c excerpt and beginning of user/cat.c

```
}

int
main(int argc, char *argv[])
{
  int fd, i;

  if (argc <= 1) {
    cat(0);
[nnorian@nnorian xv6-riscv]$
```

Figure 3.9 The main() function of user/cat.c

The list of system calls available to programs is declared in `user/user.h`, and the kernel handlers were located with grep (Figure 3.10).

```
[nnorian@nnorian xv6-riscv]$ cat user/user.h
#define SBRK_ERROR ((char *)-1)
struct stat;

// system calls
int fork(void);
int exit(int) __attribute__((noreturn));
int wait(int *);
int pipe(int *);
int write(int, const void *, int);
int read(int, void *, int);
int close(int);
int kill(int);
int exec(const char *, char **);
int open(const char *, int);
int mknod(const char *, short, short);
int unlink(const char *);
int fstat(int fd, struct stat *);
int link(const char *, const char *);
int mkdir(const char *);
int chdir(const char *);
int dup(int);
int getpid(void);
char *sys_sbrk(int, int);
int pause(int);
int uptime(void);
int sync(void);
   ... (output shortened)
[nnorian@nnorian xv6-riscv]$ grep -n "sys_read\|sys_write" kernel/sysfile.c | head
69:sys_read(void)
83:sys_write(void)
```

Figure 3.10 System call declarations and the location of their kernel handlers

Observations. (1) `user/cat.c` uses the following system calls:

- `open` – opens a file by name and returns a file descriptor;
- `read` – copies bytes from a file descriptor into the program's buffer, returning 0 at the end of the file;
- `write` – copies bytes from the buffer to a file descriptor (here, standard output);
- `close` – releases the file descriptor;
- `exit` – ends the process and returns a status to its parent.

(2) `sys_read` is implemented in `kernel/sysfile.c`, line 69 (with `sys_write` on line 83). It fetches the call's arguments from the trapped process, checks that the file descriptor is valid, and hands the actual work to `fileread()`.

(3) Code in `kernel/` runs in the privileged (supervisor) mode with full access to the hardware and to all memory, while each program in `user/` runs in its own isolated address space and can reach files and devices only by asking the kernel through system calls.

3.4 Writing the first xv6 program

A new user program, `sleep`, was created in `user/sleep.c` (Figure 3.11). It checks that exactly one argument is given; otherwise it prints `usage: sleep <ticks>` to standard error (file descriptor 2) and exits with status 1. With a valid argument it converts the text to a number with `atoi()` and passes it to the sleep system call, so the kernel suspends the process for that many clock ticks, after which the program exits with status 0.



Figure 3.11 Source code of user/sleep.c

To include the program in the build, the line `$U/_sleep\` was added to the `UPROGS` list in the Makefile, with the same tab indentation and trailing backslash as the neighbouring lines (Figure 3.12). The _ prefix is the name of the linked user binary, which `mkfs` copies into the file system image as `sleep`.

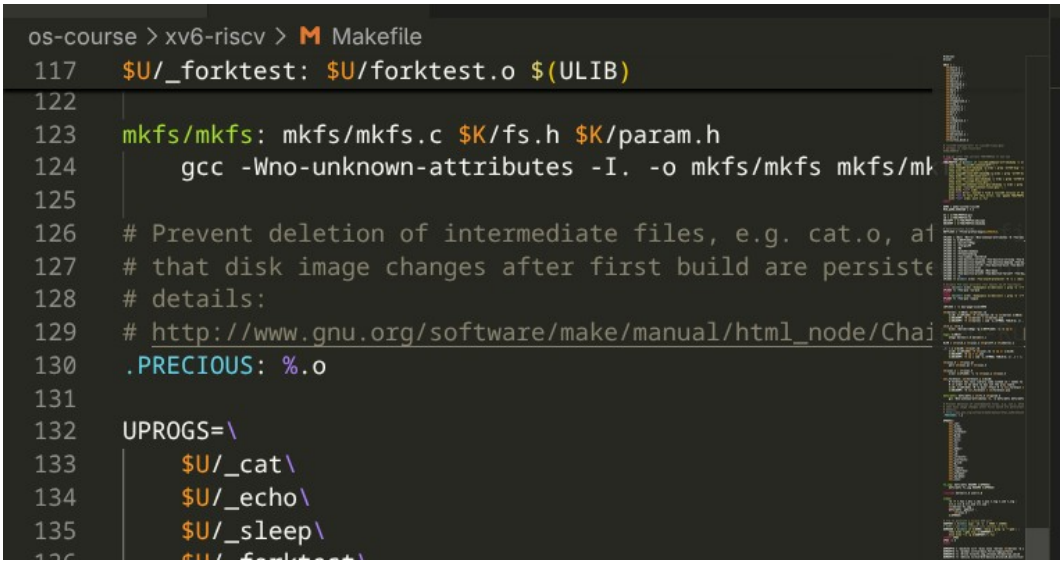


Figure 3.12 Adding the sleep program to the UPROGS list in the Makefile

xv6 was rebuilt and booted again with `make qemu`, and the program was tested (Figure 3.13). `sleep 10` paused for 10 ticks and returned to the prompt, and `sleep` without an argument printed the usage message. The program therefore makes a real system call into a kernel compiled from source, running on emulated RISC-V hardware.

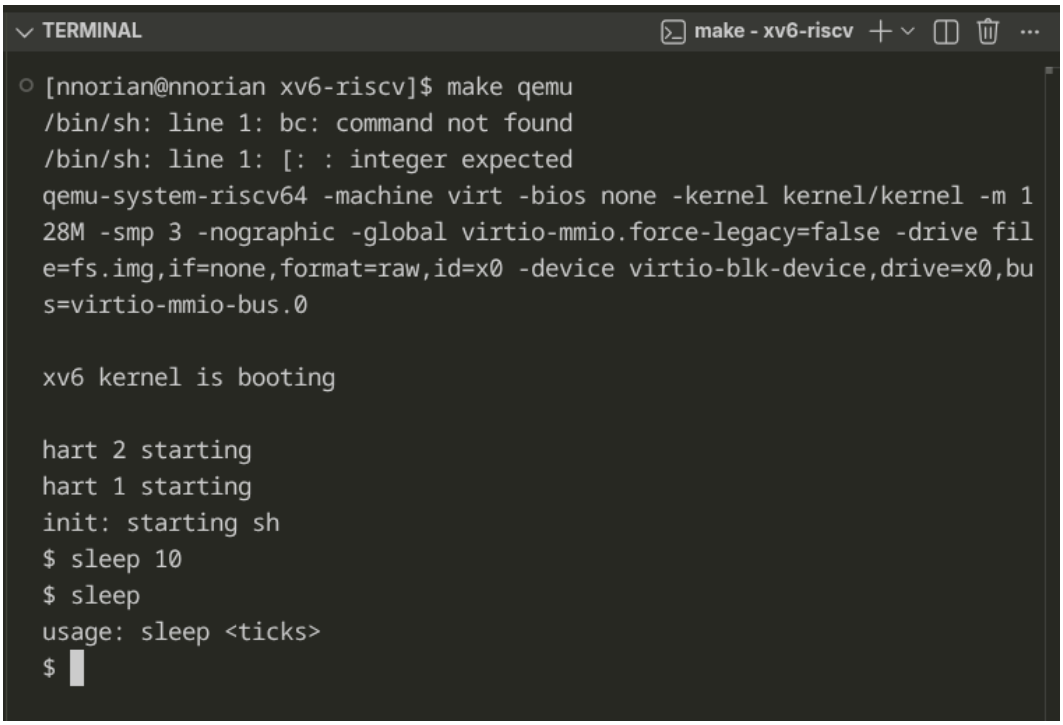


Figure 3.13 Testing the sleep program inside xv6

CONCLUSIONS

This laboratory work showed the operating system as the manager of four main groups of resources: files, processes, memory and devices. File management was visible with `ls -l`, which showed each file's owner, group and permission bits and how `chmod` changes them. Process management was visible with `ps aux` and `top`, which listed about 482 processes starting with `systemd` as PID 1, and through /proc, where the kernel exposes the state of every process. Memory management was visible with `free -h`, which showed how the OS divides 30 GiB of RAM and 4 GiB of compressed zram swap. Device management was visible with `lsblk` and `df -h`, which showed the SSD hidden behind file-like entries such as `/dev/nvme0n1`.

Building and booting xv6 moved the view from the outside to the inside of an OS. The clear split between `user/` and `kernel/` made the system call boundary concrete: a program such as `cat` only asks, through `open`, `read` and `write`, and the kernel answers in functions such as `sys_read` in `kernel/sysfile.c`. Writing the `sleep` program and adding it to the build demonstrated the full path from a user program to a kernel service and prepares the ground for modifying the kernel side in the following labs.

BIBLIOGRAPHY

1. **Operating Systems, FCIM / FAF, UTM.** Laboratory guide 1 “The OS as a Resource Manager” and Laboratory guide 2 “Meet the OS You Will Build”, 2026–2027.
2. **Cox R., Kaashoek F., Morris R.** xv6: a simple, Unix-like teaching operating system (RISC-V version). MIT PDOS, https://pdos.csail.mit.edu/6.1810/.
3. **MIT PDOS.** xv6-riscv source repository, https://github.com/mit-pdos/xv6-riscv.
4. **The Linux man-pages project.** proc(5), ps(1), top(1), free(1), lsblk(8), mount(8), https://man7.org/linux/man-pages/.